
PROGRAMMING

PROJECT - SOLVING THE TRAVELLING SALESMAN PROBLEM

E. Foussard

M1 AI, UGA

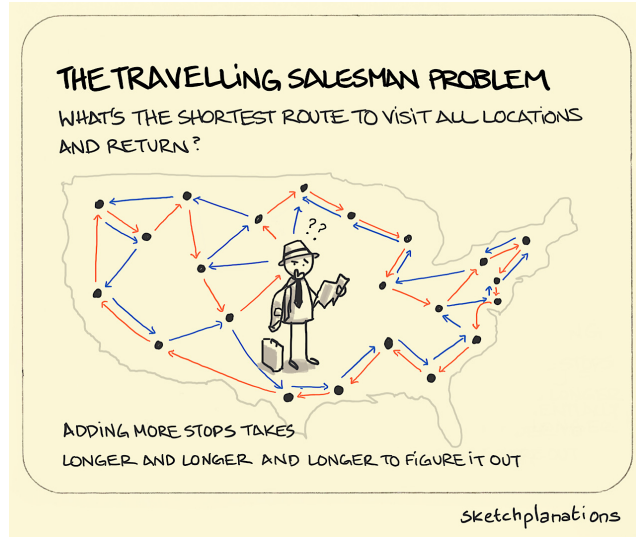


Figure 1: Image: Jono Hey, Sketchplanations

1 The Travelling Salesman Problem (TSP)

The Travelling Salesman Problem (TSP) is a very well-known combinatorial optimization problem, that can be summarized as follows: Given a list of cities and the distance between each city, what is the shortest tour that visits each city exactly once before returning to the original city. More formally, the problem consists in finding an hamiltonian cycle in a weighted graph $G = (V, E, w)$ of minimal weight.

This problem, as well as many of its variants, is classified as NP-hard, meaning that, barring a major breakthrough in complexity theory, no efficient (i.e., polynomial-time) algorithm exists to find the optimal solution of this problem. The existing methods can essentially be sorted into two categories: computationally intensive exact methods that find the optimal solution with exponential

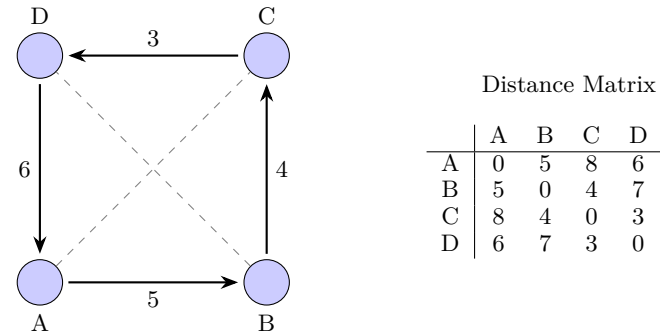


Figure 2: Example of a TSP instance with 4 cities (A, B, C, D), their distances, and a possible tour (A → B → C → D → A) of total length 18.

time complexity, and fast heuristic methods which aim at finding decent solutions (but non-necessarily optimal) in polynomial time. The problem's reach extends far beyond theory, with practical applications in transportation, logistics, manufacturing, DNA sequencing, robotics, and more. Its economic impact is substantial, making the search for effective solutions a critical challenge.

We will focus more specifically on a sub-problem of the TSP, known as the metric TSP, which has the following characteristic: $G = (V, E, w)$ satisfies the triangular inequality, i.e.

$$\forall(a, b, c) \in V^3, w(a, c) \leq w(a, b) + w(b, c)$$

We will also assume that G is complete, i.e., there exists an edge between each pair of vertices.

The goal of the project is to develop a program that solves our TSP sub-problem.

2 Project specifications

This section outlines the main requirements for the project. You are encouraged to add extra features, but first ensure that all the following specifications are correctly implemented. For example, you may add more options to your program, but the usage described below must remain functional.

2.1 Compilation and Execution

2.1.1 Compiling the project

Project sources must be provided in a standard archive format (.zip or .tar.gz). At the project root, a bash script `build-TSPsolver.sh` allows you to build the project with a single command.

```
bash build-TSPsolver.sh
```

This command compiles the project into a **build** subfolder which you contain the main executable **TSPsolver** and the test executable **tests**.

2.1.2 Basic usage

The **TSPsolver** executable takes two mandatory positional arguments:

- The relative path to the instance file
- The relative path to the output file

After compilation, running the following command should run the solver with the default method on the specified instance and write the resulting tour to the output file:

```
./build/TSPsolver <instance_path> <output_path>
```

2.1.3 Selecting the solving method

The **TSPsolver** executable also accepts an optional argument **-m**, to choose the solving method. At minimum, the following options must be available:

- **trivial** : first trivial method
- **held-karp** : Held-Karp dynamic programming method

At least another method of your choice should also be implemented. More details on the algorithms are provided in section 2.4. With the algorithm option, the usage is the following:

```
./build/TSPsolver <instance_path> <output_path> -m <algorithm>
```

2.1.4 Running the tests

The following command should run all the tests:

```
./build/tests
```

2.2 Instance files

We provide a set of problem instances of various sizes in the archive **TSP-instances.tar.gz**. These instances are sourced from **TSPlib**¹, a public library of sample instances for the Traveling Salesman Problem (TSP) and related problems. The complete specifications of the file format are available at: <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf>.

While we do not expect your program to support every data format available in **TSPlib**, it should support all file formats included in the provided archive. Each file is composed of a specification part and a data part.

¹<http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/>

Specification part

All entries in this section follow the format:

<keyword> : <value>

Every file contains at least the following entries: **NAME**, **TYPE**, and **DIMENSION**. These refer, respectively, to the instance name, the problem type (which, in our case, is always TSP), and the number of vertices.

The data section begins with a keyword indicating the representation of the instance data. The end of the data section is marked by the keyword **EOF**, which also signifies the end of the file. For this project, we consider only two data representations.

Node Coordinates In this format, the data is represented as a list of 2D coordinates. The data section starts with the keyword **NODE_COORD_SECTION**. Each line is formatted as:

<index> <x_coordinate> <y_coordinate>

Here, **<index>** is an integer, and **<x_coordinate>** and **<y_coordinate>** are floating-point numbers.

Lower Triangle of the Weight Matrix In this format, the data is represented by the lower triangle of the graph's weight matrix. The data section starts with the keyword **EDGE_WEIGHT_SECTION**, and the specification part must include the entry **EDGE_WEIGHT_FORMAT: LOWER_DIAG_ROW**. All weights are integers, and the triangular matrix is flattened into a list of coefficients. The matrix structure can be reconstructed using the size specified by the **DIMENSION** entry.

2.3 Solution files

The solution files are also composed of a specification part and a data part. The specification part is always as follows:

NAME : <name>
TYPE : TOUR
DIMENSION : <number of cities>

The data part is delimited by the keyword **TOUR_SECTION**. The tour is described as a permutation list, representing the order in which vertices are visited in the tour. The index in the list corresponds to the position in the tour, and the value at each index is the vertex to be visited at that step. We add the delimiter **-1** at the end of the permutation list, and then the keyword **EOF** to denote the end of the file.

For instance, assume that we have the following file:

NAME : example.tour
TYPE : TOUR
DIMENSION : 5

```

1
3
5
2
4
-1
EOF

```

The corresponding tour is $1 \rightarrow 3 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 1$ (returning to the start).

2.4 Algorithms

2.4.1 Trivial method

The purpose of the trivial method is to test that everything is in place and working before implementing more complex methods. It simply returns the tour consisting of the vertices sorted by there indices.

For example, for the following instance:

```

NAME: example.tsp
TYPE: TSP
DIMENSION: 5
NODE_COORD_SECTION
1 565.0 575.0
2 25.0 185.0
3 345.0 750.0
4 945.0 685.0
5 845.0 655.0

```

The trivial method should return the following tour: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1$.

2.4.2 Held-Karp algorithm

The Held-Karp algorithm is currently the most efficient exact method for solving the TSP, and it is based on dynamic programming. Since a tour is symmetric object, we can arbitrarily fix v_1 as the starting vertex without loss of generality.

We define the function $c(v_i, S)$ as the length of the shortest path between v_1 and another vertex v_i visiting each vertex in the subset $S \subseteq V$ exactly once.

Using this function, the length of the shortest tour can be expressed as follows:

$$OPT = \min_{v_i \in V \setminus \{v_1\}} \{c(v_i, V \setminus \{v_1\}) + w(v_i, v_1)\}$$

Indeed, $c(v, V \setminus \{v_1\})$ represents the length of the shortest path going from v_1 to v passing through every other vertex of the graph exactly once. To complete the tour, we simply return to v_1 from v_i .

The function $c(v_i, S)$ can be computed for any subset of vertices $S \subseteq V \setminus \{v_1\}$ and any vertex $v_i \in S$ using the following Bellman equation:

$$c(v_i, S) = \begin{cases} w(v_1, v_i) & \text{if } |S| = 1 \\ \min_{v_j \in S \setminus \{v_i\}} \{c(v_j, S \setminus \{v_i\}) + w(v_j, v_i)\} & \text{otherwise} \end{cases}$$

This equation allows us to build up the solution incrementally, computing the shortest paths for increasingly larger subsets of vertices until we obtain the full tour. The resulting algorithm is as follows:

Algorithm 1 Held-Karp Algorithm for TSP

```

1: Let  $n$  be the number of vertices
2: for  $k = 2$  to  $n$  do
3:    $c(k, \{k\}) = w(1, k)$ 
4: end for
5: for  $m = 2$  to  $n$  do
6:   for all  $S \subseteq \{2 \dots n\}, |S| = m$  do
7:     for all  $k \in S$  do
8:        $c(k, S) = \min_{i \in S \setminus \{k\}} (c(i, S \setminus \{k\}) + d(i, k))$ 
9:     end for
10:   end for
11: end for
12:  $opt = \min_{k \neq 1} \{C(k, \{2 \dots n\}) + d(k, 1)\}$ 
13: return  $opt$ 

```

This algorithm has a time complexity of $O(n^2 2^n)$ and a space complexity $O(n 2^n)$, as it requires to store the value of $c(k, S)$ for every vertex k and subset S . In order to retrieve the optimal tour, it is necessary to backtrack in the dynamic programming table.

2.4.3 Other algorithms

You are expected to implement at least one additional algorithm beyond the required ones. You are welcome to design your own method. Your work will be assessed on the **quality of your implementation and design choices**, not on how competitive your algorithm is. Before you start coding, discuss your idea with your teacher.

Alternatively, you can try to implement some state-of-the-art heuristics such as:

- 2-OPT : Start with an initial feasible solution, then iteratively improve it by swapping pairs of vertices until no further improvements are possible.
- Metaheuristics such Ant Colony Optimization, Simulated Annealing, Genetic Algorithm ...

This list is non-exhaustive, This list is not exhaustive. For more ideas, see the TSP wikipedia page: https://en.wikipedia.org/wiki/Travelling_salesman_problem.

2.5 Performance evaluation

You are expected to evaluate and compare the performance of the implemented algorithms. This can take various forms, such as:

- Measuring the execution time of each algorithm on different instances.
- Comparing the quality of the solutions obtained by each algorithm (i.e., the length of the tour).
- Analyzing the scalability of each algorithm as the size of the instances increases.
- Analyzing the behavior of the algorithms on specific instances using visualizations tools.

You can use the provided instances and also create your own instances to test the algorithms under different conditions. We provide a Python script `instance_generator.py` that generates random TSP instances. Feel free to modify it to create instances that suit your testing needs.

The results of your performance evaluation should be documented in a **report** or a **notebook**, including any relevant figures, graphs or tables to illustrate your findings.

3 Project progression and assessment method